

Formation au Langage Kotlin

Sommaire

1. Les opérations	2
1.1. Opérations logiques	2
1.2. Les entiers en action	4
1.3. Les flottants	7
1.4. Les opérateurs de comparaison	8
2. Structures de contrôle	10
2.1. Structures conditionnelles avec <code>if</code>	10
2.2. Structures conditionnelles avec <code>when</code>	13
2.3. Structures itératives avec <code>while</code> et <code>do ... while</code>	15
2.4. Structures itératives avec la boucle <code>for</code>	16
3. Listes mutables	17
3.1. Introduction	17
3.2. Travailler avec les listes mutables	19
3.3. Les listes à N dimensions	21
3.4. Boucles <code>for</code> sur les listes	24

Avant propos

Ce document présente le parcours de formation qui s'articule sur 3 types d'activités :

- **Introduction au concept :**

- Vous serez invité à lire via un lien une explication des concepts et savoirs abordés. L'ensemble des sources proviennent du site de formation *JetBrains Academy* (<https://hyperskill.org/tracks>)

Knowledge map

Topics for track **Kotlin Core**: 180 Need to learn 0 Skipped 78 Learned

for track **for project**

Math
9 topics (7 Need to learn, 0 Skipped, 2 Learned)

Computer science
249 topics (173 Need to learn, 0 Skipped, 76 Learned)

- **Réalisation de QCM :**
 - Vous serez invité à vous réaliser pour chaque partie un ou plusieurs QCM via l'ENT.
- **Application au codage :**
 - Vous serez invité à réaliser ou corriger des petits programmes Kotlin en utilisant l'IDE *IntelliJ*.

1. Les opérations

1.1. Opérations logiques

- **Exposé des concepts & savoirs :**
 - Lire le contenu proposé par JetBrains Academy : <https://hyperskill.org/learn/step/4515>
- **Validation de la compréhension par QCM :**
 - Réaliser le QCM de l'ENT/Pronote : LIEN_QCM_A_DEPOSER
- **Mise en application :**

Problème 14_1-a : Invitations et cadeaux

Enoncé :

1. Groundhog organise une fête. Il est connu pour son pragmatisme, donc seules les personnes qui apportent un cadeau seront autorisées à y assister. Et, bien sûr, un invité doit avoir une invitation.

Écris un programme qui vérifie si un invité doit être autorisé à entrer à la fête. Le programme doit lire **deux booléens**, chacun sur une **ligne séparée** :

- Le premier indique si l'invité a une invitation ;
- Le second indique si l'invité a apporté un cadeau.

Affiche **true** si les deux conditions sont remplies et **false** sinon.

Exemple 1 :

Input :

```
true
true
```

Output :

```
true
```

Exemple 2 :

Input :

```
true
false
```

Output :

```
false
```

Fichier de travail :

```
.\Learn_Kotlin_LdV\01-EXERCICES\14-Les_operations\14_1-Operations_logiques\P14_a-
Invitations_et_cadeaux
```

Problème 14_1-b : Coder une expression booléenne

Enoncé :

Écris un programme qui lit trois variables booléennes **x**, **y** et **z** (chacune sur une ligne séparée), puis affiche le résultat de l'expression logique suivante : **NOT(x AND y) OR (z)**.

Exemple 1 :

Input :

```
true
true
```

false

Output :

false

Exemple 2 :

Input :

true
false
true

Output :

true

Fichier de travail :

. \Learn_Kotlin_LdV\01-EXERCICES\14-Les_operations\14_1-Operations_logiques\P14_1_b-Coder_expression_bool

1.2. Les entiers en action

- **Exposé des concepts & savoirs :**

- Lire le contenu proposé par **JetBrains Academy** : <https://hyperskill.org/learn/step/4488>

- **Validation de la compréhension par QCM :**

- Réaliser le QCM de l'ENT/**Pronote** : LIEN_QCM_A_DEPOSER

- **Mise en application :**

Problème 14_4-a : Partager équitablement des noisettes

Enoncé :

N écureuils ont trouvé K noisettes et ont décidé de les partager équitablement. Calcule combien de noisettes revient à chaque écureuil.

Format des données en entrée : Deux nombres positifs N et K, chacun n'étant pas supérieur à 10 000.

Input

3
14

Output

4

Fichier de travail :

.\\Learn_Kotlin_LdV\\01-EXERCICES\\14-Les_operations\\14_4-Entiers_en_action\\P14_4_a-Partager_noisettes

Problème 14_4-b : Noisettes restantes

Enoncé :

N écureuils ont trouvé **K** noisettes et ont décidé de les partager équitablement. Détermine combien de noisettes resteront après que chaque écureuil a pris une quantité égale de noisettes.

Format des données en entrée : Deux nombres positifs **N** et **K**, chacun n'étant pas supérieur à 10 000.

Input

3
14

Output

2

Fichier de travail :

.\\Learn_Kotlin_LdV\\01-EXERCICES\\14-Les_operations\\14_4-Entiers_en_action\\P14_4_b-Noisettes_restantes

Problème 14_4-c : Nombre pair suivant

Enoncé :

Étant donné un nombre naturel **N**, n'étant pas supérieur à 10 000. Affiche le nombre pair qui suit ce **N**.

Astuce : Essaie d'écrire une formule courte en utilisant l'opérateur modulo %.

Exemple 1 :

Input

7

Output

8

Exemple 2 :

Input

8

Output

10

Fichier de travail :

`.\Learn_Kotlin_LdV\01-EXERCICES\14-Les_operations\14_4-Entiers_en_action\P14_4_c-
Nombre_pair_suivant`

Problème 14_4-d : Somme des chiffres

Enoncé :

On donne un entier à trois chiffres (un entier compris entre 100 et 999). Trouve la somme de ses chiffres.

N'oublie pas que tu peux obtenir les chiffres d'un nombre avec `/10` et `%10`.

Exemple :

Input

476

Output

17

Fichier de travail :

`.\Learn_Kotlin_LdV\01-EXERCICES\14-Les_operations\14_4-Entiers_en_action\P14_4_d-
Somme_des_chiffres`

Problème 14_4-e : Formatage de dates

Enoncé :

On te donne un nombre qui représente le nombre de secondes écoulées depuis le **1/1/1970**.

Écris un programme qui calcule l'heure actuelle, c'est-à-dire qui trouve les heures, minutes et secondes correspondant au nombre de secondes donné.

Format : heures:minutes:secondes

Exemple : 14:9:7

Tu n'as rien à importer, utilise simplement les opérateurs % et /, et souviens-toi de la durée d'une journée. Tu n'as pas besoin d'afficher le nombre de jours, et ne tiens pas compte de l'UTC ou du GMT.

Exemple 1 :

Output

15:47:4

Fichier de travail :

.\Learn_Kotlin_LdV\01-EXERCICES\14-Les_operations\14_4-Entiers_en_action\P14_4_e-Formatage_date

1.3. Les flottants

- **Exposé des concepts & savoirs :**

- Lire le contenu proposé par **JetBrains Academy** : <https://hyperskill.org/learn/step/4502>

- **Validation de la compréhension par QCM :**

- Réaliser le QCM de l'ENT/**Pronote** : LIEN_QCM_A_DEPOSER

- **Mise en application :**

Problème 14_5-a : Evaluer une expression

Enoncé :

Écris un programme qui lit quatre valeurs réelles **a**, **b**, **c**, **d** et calcule l'expression suivante :

- $a * 10.5 + b * 4.4 + (c + d) / 2.2$

Exemple 1 :

Input :

1
2.5
0
4.4

Output :

23.5

Fichier de travail :

.\Learn_Kotlin_LdV\01-EXERCICES\14-Les_operations\14_5-Les_flottants\P14_5_a-Evaluer_expression

Problème 14_5-b : Celsius vers Fahrenheit

Enoncé :

Écris un programme qui lit une température en degrés Celsius (°C) et affiche son équivalent en degrés Fahrenheit (°F).

Utilise cette formule : $* \text{fahrenheit} = \text{celsius} * 1.8 + 32$

Exemple 1 :

Input :

32.9

Output :

91.22

Exemple 2 :

Input :

0

Output :

32.0

Fichier de travail :

.\Learn_Kotlin_LdV\01-EXERCICES\14-Les_operations\14_5-Les_flottants\P14_5_b-Celsius_vers_Fahrenheit

1.4. Les opérateurs de comparaison

- **Exposé des concepts & savoirs :**

- Lire le contenu proposé par JetBrains Academy : <https://hyperskill.org/learn/step/4525>

- Validation de la compréhension par QCM :
 - Réaliser le QCM de l'ENT/**Pronote** : LIEN_QCM_A_DEPOSER
- Mise en application :

Problème 14_6-a : Vérifier la valeur

Enoncé :

Écris un programme qui lit une valeur et vérifie si elle est inférieure à 10 mais supérieure à 0.

Exemple :

Input :

0

Output :

false

Fichier de travail :

.\Learn_Kotlin_LdV\01-EXERCICES\14-Les_operations\14_6-Operateurs_comparaison\P14_6_a-Verifier_valeur

Problème 14_6-b : Tailles croissantes ou décroissantes

Enoncé :

Imagine qu'il y a trois garçons dans un cours de sport.

Écris un programme qui vérifie si les garçons sont disposés dans l'ordre croissant ou décroissant de leur taille. Le programme doit lire trois entiers **h1**, **h2**, **h3** et afficher **true** ou **false**.

Si les garçons ont la même taille, on considère qu'ils sont disposés correctement.

Exemple 1 :

Input :

165
161
158

Output :

true

Exemple 2 :

Input :

155
165
160

Output :

false

Exemple 3 :

Input :

161
161
165

Output :

true

Fichier de travail :

.\Learn_Kotlin_LdV\01-EXERCICES\14-Les_operations\14_6-Operateurs_comparaison\P14_6_b-Tailles_croissantes_decroissantes

2. Structures de contrôle

2.1. Structures conditionnelles avec **if**

- **Exposé des concepts & savoirs :**
 - Lire le contenu proposé par JetBrains Academy : <https://hyperskill.org/learn/step/4625>
- **Validation de la compréhension par QCM :**
 - Réaliser le QCM de l'ENT/**Pronote** : LIEN_QCM_A_DEPOSER
- **Mise en application :**

Problème 15_1-a : Positif, négatif ou nul

Énoncé :

Écris un programme qui lit un entier depuis la console et affiche :

- si le nombre < 0 : négatif ;
- si le nombre > 0 : positif ;
- si le nombre $= 0$: zéro.

Exemple 1 :

Input :

-5

Output :

negative

Fichier de travail :

.\\Learn_Kotlin_LdV\\01-EXERCICES\\15-Structures_de_controle\\15_1-if\\P15_1_a-Positif_negatif_nul

Problème 15_1-b : Sommeil et santé

Énoncé :

Ann a regardé une émission de télévision sur la santé et a appris qu'elle devrait dormir au moins **A** heures par jour, mais que trop dormir n'est pas sain non plus : il ne faut pas dormir plus de **B** heures. Actuellement, Ann dort **H** heures par jour. Si le programme de sommeil d'Ann respecte les recommandations de l'émission, affiche "**Normal**". Si Ann dort moins de **A** heures, affiche "**Deficiency**". Si elle dort plus de **B** heures, affiche "**Excess**".

Entrée : Les entrées du programme sont trois chaînes, dans l'ordre : **A**, **B**, **H**. On suppose que **A** est toujours inférieur ou égal à **B**.

Remarque sur la casse : Respecte exactement la casse demandée dans l'énoncé. Par exemple, si le programme doit afficher "**Excess**", alors "**excess**", "**EXCESS**", "**ExCeSs**" ou autres variantes ne seront pas considérées comme correctes.

Conseil : Réfléchis **attentivement** à toutes les conditions à utiliser. Accorde une attention particulière au caractère strict des opérateurs conditionnels : distingue bien $<$ et $\leq$; $>$ et $\geq$. Pour savoir lesquels employer, relis soigneusement l'énoncé.

Exemple 1 :

Input :

6
10
8

Output :

Normal

Exemple 2 :

Input :

7
9
10

Output :

Excess

Exemple 3 :

Input :

7
9
2

Output :

Deficiency

Fichier de travail :

.\\Learn_Kotlin_LdV\\01-EXERCICES\\15-Structures_de_controle\\15_1-if\\P15_1_b-Sommeil_et_sante

Problème 15_1-c : Nombre de diviseurs

Enoncé :

Écris un programme qui lit un entier et vérifie s'il est divisible par 2, 3, 5 ou 6.

Si le nombre est divisible par **M**, le programme doit afficher "**Divided by M**".

Le programme doit vérifier tous les diviseurs indiqués ci-dessus. L'ordre des diviseurs dans le

résultat peut être quelconque.

Astuce : Utilise l'opérateur modulo `%` pour vérifier si un nombre est divisible par un autre.

Exemple 1 :

Input :

6

Output :

Divided by 2
Divided by 3
Divided by 6

Fichier de travail :

`.\Learn_Kotlin_LdV\01-EXERCICES\15-Structures_de_controle\15_1-if\P15_1_c-
nombre_diviseurs`

2.2. Structures conditionnelles avec **when**

- **Exposé des concepts & savoirs :**

- Lire le contenu proposé par **JetBrains Academy** : <https://hyperskill.org/learn/step/4631>

- **Validation de la compréhension par QCM :**

- Réaliser le QCM de l'ENT/**Pronote** : LIEN_QCM_A_DEPOSER

- **Mise en application :**

Problème 15_2-a : Harry Potter

Enoncé :

Harry Potter a besoin de notre aide pour identifier les traits qui caractérisent chaque maison.

Lis une chaîne qui représente une maison et affiche le résultat suivant :

- si c'est **gryffindor**, affiche **bravery** ;
- si c'est **hufflepuff**, affiche **loyalty** ;
- si c'est **slytherin**, affiche **cunning** ;
- si c'est **ravenclaw**, affiche **intellect** ;
- sinon, affiche **not a valid house**.

Exemple 1 :

Input :

gryffindor

Output :

bravery

Fichier de travail :

.\Learn_Kotlin_LdV\01-EXERCICES\15-Structures_de_controle\15_2-when\P15_2_a-Harry_Potter

Problème 15_2-b : Direction

Enoncé :

Écris un programme qui lit un numéro de direction :

- 1 → "move up"
- 2 → "move down"
- 3 → "move left"
- 4 → "right"
- 0 → "do not move"

Si l'entrée ne correspond à aucune des directions listées, affiche "**error!**".

Remarque : Le texte en sortie doit correspondre exactement à l'exemple donné, y compris la casse des lettres et les espaces.

Exemple 1 :

Input :

1

Output :

move up

Fichier de travail :

.\Learn_Kotlin_LdV\01-EXERCICES\15-Structures_de_controle\15_2-when\P15_2_b-Direction

2.3. Structures itératives avec **while** et **do ... while**

- Exposé des concepts & savoirs :

- Lire le contenu proposé par JetBrains Academy : <https://hyperskill.org/learn/step/4659>

- Validation de la compréhension par QCM :

- Réaliser le QCM de l'ENT/**Pronote** : LIEN_QCM_A_DEPOSER

- Mise en application :

Problème 15_4-a : Calcul des intérêts

Enoncé :

Lorsqu'une banque rencontre des difficultés financières, le gouvernement peut rembourser le dépôt d'un client si celui-ci est inférieur à **700 000**. Le taux d'intérêt pour un dépôt particulier est de **7,1 %** par an. Les intérêts sont ajoutés au dépôt à la fin de l'année, puis un nouveau calcul est effectué sur la somme mise à jour.

Détermine **combien d'années** il faudra pour que la somme du dépôt dépasse la valeur protégée par le gouvernement.

Format des données en entrée : La somme initiale du dépôt.

Il est garanti que la valeur se situe entre 50 000 et 700 000.

Remarque : Le montant du dépôt augmente chaque année d'une valeur proportionnelle au taux d'intérêt, ici 1,071 (soit 7,1 %). Ainsi, la formule de calcul du montant du dépôt est la suivante : $\text{deposit} = \text{deposit} * 1.071$

Exemple 1 :

Input :

650000

Output :

2

Fichier de travail :

.\\Learn_Kotlin_LdV\\01-EXERCICES\\15-Structures_de_controle\\15_4-while_ET_do_while\\P15_4_a-Calcul_interets

Problème 15_4-b : Somme de valeurs

Enoncé :

Calcule la somme de tous les éléments d'une séquence, se terminant par 0.

Le nombre 0 lui-même n'est pas inclus dans la séquence et sert uniquement de marqueur de fin.

Cependant, si tu ajoutes 0 à la somme, rien de grave ne se produira :)

Exemple 1 :

Input :

```
3
6
8
0
```

Output :

```
17
```

Fichier de travail :

```
.\Learn_Kotlin_LdV\01-EXERCICES\15-Structures_de_controle\15_4-while_ET_do_while\P15_4_b-Somme_valeurs
```

2.4. Structures itératives avec la boucle **for**

- **Exposé des concepts & savoirs :**

- Lire le contenu proposé par JetBrains Academy : <https://hyperskill.org/learn/step/4652>

- **Validation de la compréhension par QCM :**

- Réaliser le QCM de l'ENT/**Pronote** : LIEN_QCM_A_DEPOSER

- **Mise en application :**

Problème 15_5-a : Somme des entiers d'un intervalle

Enoncé :

Lis deux entiers **a** et **b**. Affiche la somme de tous les entiers de **a** à **b** inclus.

Il est garanti que **a** < **b**.

Exemple 1 :

Input :

```
1
24
```

Output :

300

Fichier de travail :

.\Learn_Kotlin_LdV\01-EXERCICES\15-Structures_de_controle\15_5-Boucle_for\P15_5_a-Somme_intervalle

Problème 15_5-b : Valeur minimale de N nombres

Enoncé :

Écris un programme qui trouve la valeur minimale parmi N nombres.

La première ligne contient le nombre N. Les lignes suivantes contiennent les N nombres.

Affiche un entier correspondant au minimum des N nombres.

Exemple 1 :

Input :

5
5
1
4
2
3

Output :

1

Fichier de travail :

.\Learn_Kotlin_LdV\01-EXERCICES\15-Structures_de_controle\15_5-Boucle_for\P15_5_b-Minimal_de_N_nombres

3. Listes mutables

3.1. Introduction

- **Exposé des concepts & savoirs :**
 - Lire le contenu proposé par JetBrains Academy : <https://hyperskill.org/learn/step/14879>
- **Validation de la compréhension par QCM :**

- Réaliser le QCM de l'ENT/**Pronote** : LIEN_QCM_A_DEPOSER

- **Mise en application :**

Problème 16_1-a : Initialisation d'une liste d'entiers

Enoncé :

Crée une liste mutable d'entiers (**Int**) nommée **numbers** avec cinq éléments : **12**, **17**, **8**, **101** et **33**. Ensuite, le code affichera cette liste.

Utilise le modèle de code fourni.

Output :

```
12, 17, 8, 101, 33
```

Fichier de travail :

```
.\Learn_Kotlin_LdV\01-EXERCICES\16-Listes_mutable\16_1-Introduction\P16_1-a-  
Initialisation_liste_entiers
```

Problème 16_1-b : Initialisation d'une liste

Enoncé :

Crée une liste mutable d'entiers (**Int**) nommée **numbers** contenant 100 éléments.

- Le premier élément doit être 1,
- le dixième élément doit être 10,
- le centième élément doit être 100.

Tous les autres éléments doivent être égaux à 0.

Utilise le modèle de code fourni.

Output :

```
1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 10, 0, 0, ... 100
```

Fichier de travail :

```
.\Learn_Kotlin_LdV\01-EXERCICES\16-Listes_mutable\16_1-Introduction\P16_1_b-Init_list
```

Problème 16_1-c : Echange

Enoncé :

Supposons que tu disposes d'une liste **numbers** de type **Int**. Échange le premier et le dernier élément de la liste.

Il est garanti que la taille de la liste est supérieure à 0.

Exemple 1 :

Input :

1 2 3 4 5

Output :

5 2 3 4 1

Fichier de travail :

`.\Learn_Kotlin_LdV\01-EXERCICES\16-Listes_mutables\16_1-Introduction\P16_1_c-Echange`

3.2. Travailler avec les listes mutables

- **Exposé des concepts & savoirs :**

- Lire le contenu proposé par JetBrains Academy : <https://hyperskill.org/learn/step/14902>

- **Validation de la compréhension par QCM :**

- Réaliser le QCM de l'ENT/Pronote : LIEN_QCM_A_DEPOSER

- **Mise en application :**

Problème 16_2-a : Encore une autre liste

Enoncé :

Crée une liste mutable de chaînes (`String`) nommée `capitals` avec cinq éléments : "Tokyo", "Moscow", "Paris", "Washington", "Beijing".

Output :

Tokyo, Moscow, Paris, Washington, Beijing

Fichier de travail :

`.\Learn_Kotlin_LdV\01-EXERCICES\16-Listes_mutables\16_2-Travailler_avec_listes_mutables\P16_2_a-Encore_une_autre_liste`

Problème 16_2-b : Addition de listes

Enoncé :

Concatène deux listes mutables `firstList` et `secondList`, puis affiche le résultat.

Dans l'exemple ci-dessous, chaque ligne correspond à une liste distincte. Les éléments sont

séparés par des espaces.

Astuce : Utilise la fonction `joinToString()`.

Exemple 1 :

Input :

```
valar morghulis  
valar dohaeris
```

Output :

```
valar, morghulis, valar, dohaeris
```

Fichier de travail :

`.\Learn_Kotlin_LdV\01-EXERCICES\16-Listes_mutables\16_2-
Travailler_avec_listes_mutables\P16_2_b-Addition_de_listes`

Problème 16_2-c : Garde au portail

Enoncé :

Tu dois aider le garde qui surveille la porte principale menant à la contrée sauvage au-delà du Mur. C'est sa responsabilité d'ajouter toute personne qui passe par la porte à la liste `backToTheWall`.

Ajoute le nom du garde (stocké dans la variable `returnedWatchman`) à la liste `backToTheWall` et affiche le résultat avec `joinToString()`.

Exemple :

Input :

```
Benjen Stark, Samwell Tarly, Gared Tuttle  
Jon Snow
```

Output :

```
Benjen Stark, Samwell Tarly, Gared Tuttle, Jon Snow
```

Fichier de travail :

`.\Learn_Kotlin_LdV\01-EXERCICES\16-Listes_mutables\16_2-
Travailler_avec_listes_mutables\P16_2_c-Garde_au_portail`

Problème 16_2-d : Jouer avec les nombres

Enoncé :

Tu disposes d'une liste mutable d'entiers `numbers`.

- Ajoute la somme de tous les éléments de la liste au début de celle-ci.
- Supprime ensuite l'avant-dernier élément de la liste.

Il est garanti que la taille de la liste est supérieure à 2.

Exemple :

Input :

```
8 11 1 2 3
```

Output :

```
25 8 11 1 3
```

Fichier de travail :

`.\Learn_Kotlin_LdV\01-EXERCICES\16-Listes_mutable\16_2-Travailler_avec_listes_mutable\P16_2_d-Jouer_avec_nombres`

3.3. Les listes à N dimensions

• Exposé des concepts & savoirs :

- Lire le contenu proposé par JetBrains Academy : <https://hyperskill.org/learn/step/15127>

• Validation de la compréhension par QCM :

- Réaliser le QCM de l'ENT/**Pronote** : LIEN_QCM_A_DEPOSER

• Mise en application :

Problème 16_3-a : Listes Kotlin

Enoncé :

Écris un code qui affiche la troisième ligne de `inputList` sous forme d'une seule chaîne. Les éléments doivent être séparés par une virgule et un espace.

Il est garanti que `inputList` est déjà initialisée et contient au moins 3 listes imbriquées.

Astuce : Utilise la fonction `joinToString()`.

Exemple 1 :

Input :

```
4
(¬¬)_ Program
_(^.) / with
(>^_)> Kotlin!
wrong row
```

Output :

```
(>^_)>, Kotlin!
```

Fichier de travail :

`.\Learn_Kotlin_LdV\01-EXERCICES\16-Listes_mutables\16_3-Listes_N_dimensions\P16_3_a-Listes_Kotlin`

Problème 16_3-b : Renversement

Enoncé :

Imagine que tu as une liste 2D `inputList`. Il est garanti qu'elle comporte au moins 2 listes imbriquées.

Crée une nouvelle liste 2D `resultList` comportant 2 sous-listes : * la première sous-liste est la dernière liste imbriquée de `inputList`, * la seconde sous-liste est la première de `inputList`.

Affiche la liste résultat `resultList`.

Astuce : Tu peux utiliser les méthodes `first()` et `last()`.

Exemple 1 :

Input :

```
3
00 01 02 03
10 11 12 13
20 21 22 23
```

Output :

```
[[20, 21, 22, 23], [00, 01, 02, 03]]
```

Fichier de travail :

`.\Learn_Kotlin_LdV\01-EXERCICES\16-Listes_mutables\16_3-Listes_N_dimensions\P16_3_b-Renversement`

Problème 16_3-c : Cube de zéros

Enoncé :

Crée une liste tridimensionnelle 3x3x3 contenant des éléments de type `Int` et remplie de zéros (0).

Affiche la liste de listes sous forme d'une seule chaîne (utilise simplement la fonction `print()` ou `println()`).

Sortie attendue :

```
[[[0, 0, 0], [0, 0, 0], [0, 0, 0]], [[0, 0, 0], [0, 0, 0], [0, 0, 0]], [[0, 0, 0], [0, 0, 0], [0, 0, 0]]]
```

Fichier de travail :

`.\Learn_Kotlin_LdV\01-EXERCICES\16-Listes_mutables\16_3-Listes_N_dimensions\P16_3-c-Cube_de_zeros`

Problème 16_3-d : Coins

Enoncé :

Imagine que tu as une liste 2D `inputList`, dont la dimension est supérieure ou égale à deux. Affiche tous ses éléments situés dans les coins dans l'ordre suivant : de gauche à droite puis de haut en bas.

Affiche le résultat de deux éléments par ligne. Utilise un espace unique pour séparer deux éléments sur la même ligne.

Regarde les exemples ci-dessous et souviens-toi que les listes imbriquées peuvent avoir des longueurs différentes !

Astuce : Tu peux utiliser `first()` et `last()`.

Exemple 1 :

Input :

```
3
1 0 1
0 0 0
1 0 1
```

Output :

```
1 1
1 1
```

Exemple 2 :

Input :

```
2
2 2
2 2
```

Output :

```
2 2
2 2
```

Fichier de travail :

.\Learn_Kotlin_LdV\01-EXERCICES\16-Listes_mutable\16_3-Listes_N_dimensions\P16_3_d-Coins

3.4. Boucles **for** sur les listes

- **Exposé des concepts & savoirs :**
 - Lire le contenu proposé par JetBrains Academy : <https://hyperskill.org/learn/step/15022>
- **Validation de la compréhension par QCM :**
 - Réaliser le QCM de l'ENT/**Pronote** : LIEN_QCM_A_DEPOSER
- **Mise en application :**

Problème 16_4-a : Nombre dans une séquence

Enoncé :

Écris un programme qui vérifie si un ensemble de **N** nombres contient un nombre **M**.

- La première ligne contient le nombre **N**.
- Les **N** lignes suivantes contiennent l'ensemble des nombres, un nombre par ligne.
- La dernière ligne contient un entier **M**.

Tu dois afficher soit **YES**, soit **NO**.

Exemple 1 :

Input :

```
3
1
3
2
```

3

Output :

YES

Fichier de travail :

.\\Learn_Kotlin_LdV\\01-EXERCICES\\16-Listes_mutable\\16_4-
Boucles_for_sur_listes\\P16_4_a-Nombre_dans_une_sequence

Problème 16_4-b : Nombre d'occurrences

Enoncé :

Écris un programme qui compte combien de fois un nombre **M** apparaît parmi **N** nombres.

- La première ligne contient le nombre **N**.
- Les **N** lignes suivantes contiennent les nombres.
- La dernière ligne contient le nombre **M**.

La sortie doit être un seul entier non négatif.

Exemple 1 :

Input :

6
1
2
3
4
2
1
2

Output :

2

Fichier de travail :

.\\Learn_Kotlin_LdV\\01-EXERCICES\\16-Listes_mutable\\16_4-
Boucles_for_sur_listes\\P16_4_b-Nombre_occurrences

Problème 16_4-c : Triples

Enoncé :

Un **triple** est une suite de trois entiers consécutifs en ordre croissant. Par exemple, 3, 4, 5 est un triple, mais 5, 4, 3 et 2, 4, 6 ne le sont pas.

- La première ligne contient la taille de la liste.
- Les lignes suivantes contiennent les éléments de la liste.

La sortie doit être un seul entier représentant le nombre de triples dans la liste.

Dans l'exemple ci-dessous, il y a deux triples : 4, 5, 6 et 5, 6, 7.

Exemple 1 :

Input :

```
6
1
2
4
5
6
7
```

Output :

```
2
```

Fichier de travail :

.\Learn_Kotlin_LdV\01-EXERCICES\16-Listes_mutables\16_4-
Boucles_for_sur_listes\P16_4_c-Triples